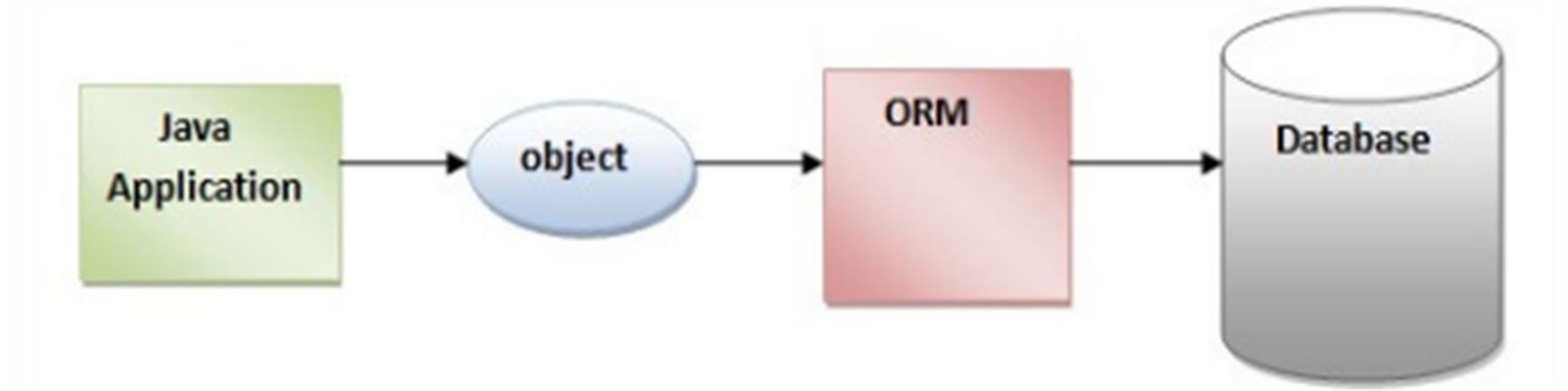
HIBERNATE FRAMEWORK

HIBERNATE FRAMEWORK

- **Hibernate is a Java framework that simplifies the development of Java application to interact with the database.**
- **It is an open source, lightweight framework.**
- **Hibernate is an ORM (Object Relational Mapping) tool.**
- **Hibernate implements the specifications of JPA (Java Persistence API) for data persistence.**

ORM TOOL
(Object Relational Mapping Tool)

- **An ORM tool simplifies the data creation, data manipulation and data access.**
- **It is a programming technique that maps the object to the data stored in the database.**



- **The ORM tool internally uses the JDBC API to interact with the database.**

JPA

- **Java Persistence API (JPA) is a Java specification(collection of classes and interfaces) that provides certain functionality and standard to ORM tools.**

- **The javax.persistence package contains the JPA classes and interfaces.**

- **It facilitates ORM tool to manage relational data in Java applications.**

- **It provides a platform to work directly with objects instead of using SQL statements.**

Advantages of Hibernate Framework

1) Open Source and Lightweight : **Hibernate framework is open source under the LGPL license and lightweight.**

2) Fast Performance : **The performance of hibernate framework is fast because cache is internally used in hibernate framework.**

3) Database Independent Query : Hibrenate supports Hibernate Query Language. HQL is the object-oriented version of SQL. It generates the database independent queries. So you don't need to write database specific queries. Before Hibernate, if database is changed for the project, we need to change the SQL query as well.

4) Automatic Database and Table Creation : Hibernate framework provides the facility to create the tables and database automatically depending on which DB we are using. So there is no need to create tables or database manually.

For creating the a hibernate application, we need to follow the following steps:

1. **Create the Persistent / Entity class.**

2. **Do the mapping for Entity class using @Entity , @Id annotations etc.**

3. **Create the Configuration file(persistence.xml) by specifying database name , username and password of database etc.**

4. **Create the class that stores or retrieves the persistent object , i.e.. Class to perform database operations.**

5. **Run the program.**

ENTITY CLASS

- **Java classes whose objects will be stored in database tables are called persistent classes in Hibernate. They are also called as Entity class.**
-
- **The entity class is a class that represents a table in a database .**
- **To mark any class as Entity class , the class must be annotated with @Entity annotation.**

Entity class must follow the following rules :

1. **A no-arg constructor:** It is recommended to have a NO argument constructor so that hibernate can create the object of the Persistent class.

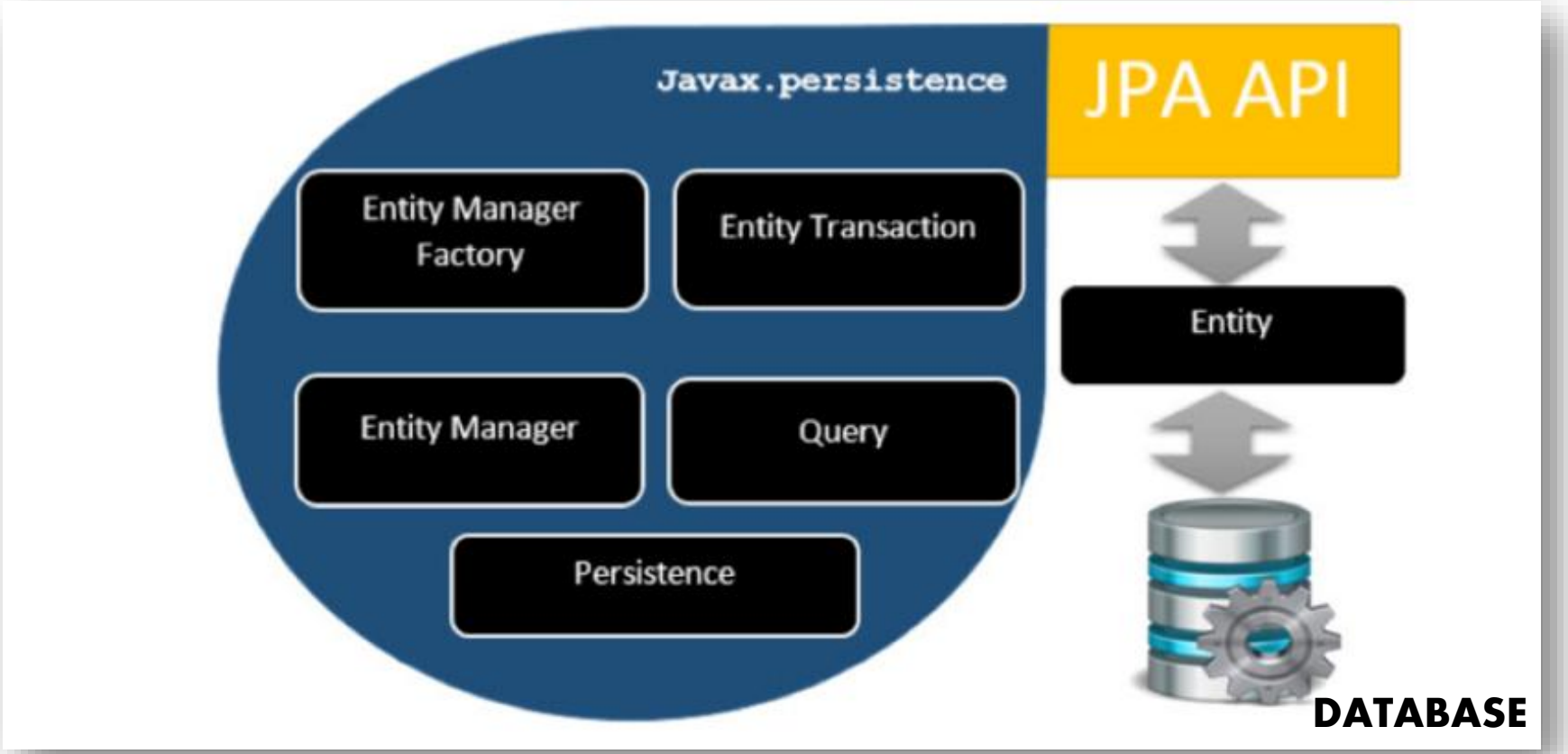
2. **Map the class :** Annotate the persistent class with @Entity annotation to map the class with the table in database.

3. **Primary key :** An attribute must be specified as a Primary key using @Id annotation. This attribute behaves as a primary key in database.

4. **Getter and setter methods:** The Hibernate recognizes the attributes by getter and setter method names by default. So getter and setter methods must be created for all the attributes.

COMPONENTS OF JPA (OR)
CLASS LEVEL ARCHITECHTURE OF JPA

- **The following image shows the class level architecture of JPA. It shows the core classes and interfaces of JPA.**



- **Persistence :** **This class contains static methods , like createEntityManagerFactory(String persistenceUnitName) to obtain EntityManagerFactory instance. It accepts the persistence unit name as input and loads all the properties from the persistence file.**

- **EntityManagerFactory :** **This is a factory class of EntityManager. It consists of createEntityManager() method creates , returns and manages multiple EntityManager instances.**

- **EntityManager :** **is the primary JPA interface. The EntityManager manages the entities and consists of methods to perform database operations such insert , update , delete and retrieve objects from database.**

- **It consists of following methods :**

  - persist(object) : **It is used to store(insert) an entity object into the database.**
  - remove(object) : **It is used to remove an entity object from the database.**
  - merge(object) : **It is used to update the existing object in the database by merging the states of given entity object.**
  - find(entityClassName.class , primaryKeyValue) : **Used to find / search for object based on primary key.**
  - getTransaction() : **creates and return an object of EntityTransaction.**
  - createQuery(String sqlString) : **It accepts a Java Persistence query language (JPQL) query as an input and creates an object of Query for executing JPQL statement.**

- **EntityTransaction :** For each EntityManager, operations are maintained by EntityTransaction. It is used to keep track of transactions performed by the EntityManager. It consists of methods such as :

    - begin() : it is used to specify that transaction has started.
    - commit() : it specifies that the transaction has ended and saves then changes to the database.

- **Query :** The object of Query can be obtained by calling the createQuery() of EntityManager interface. The Query object is used to execute the JPQL / HQL/SQL queries. For executing the queries , it consists of methods such as ,

    - int executeUpdate() : is used to execute the update or delete query.
    - List<E> getResultList() : is used to execute the select query. It returns the results as a List.

HQL (Hibernate Query Language) / JPQL (Java Persistence Query Language)

- **HQL / JPQL is an object-oriented query language similar to SQL but works with entity objects, not tables.**
- **Instead of querying database tables and columns, we query entities and their properties(variables).**
- **Hibernate translates HQL into native SQL queries internally.**

Example :

SQL - select * from employee ; // here employee is table name

HQL - select e from Employee e ; // here Employee is the entity class name.

SQL – update employee set salary=67000 where name = 'Ram';

HQL – update Employee e set e.salary =67000 where e.name = 'Ram';

STATIC HQL QUERIES

- **Written once and do not change during runtime.**
- **Easy to read and maintain.**
- **Good for fixed operations like fetching all records or specific criteria.**
- **Example : update Employee e set e.salary =67000 where e.name = 'Ram';**

DYNAMIC HQL QUERIES

- **Built dynamically at runtime based on conditions or user input.**
- **Useful when filtering/searching with variable parameters.**

Parameters in HQL

- **To prevent SQL Injection and make code flexible, we use parameters instead of directly concatenating user inputs.**
- **HQL supports two types of parameters:**

Type	Syntax Example	Description
Positional Parameters	?1, ?2, ?3	Based on position.
Named Parameters	:name	Based on parameter name (recommended).

Positional Parameters

- Represented by ? followed by index / position.
- Order matters — must be set in the same sequence.
- Example :

```
Query query = manager.createQuery("update Employee e set e.salary = 1? where e.name = 2?");

    query.setParameter(1 , 90000);

    query.setParameter(2 , "Sheela");

    Query.executeUpdate();
```

Named Parameters

- **More readable and less error-prone.**
- **Use :parameterName in the HQL query and set value using setParameter().**

- **Example :**

```
Query query = manager.createQuery("select  e from Employee e where e.name = :ename ");
    query.setParameter ( "ename" ,  Rahul );
    List<Employee> list = query.getResultList();
```

HIBERNATE PROGRAMS

- **Create an entity class.**

```java
import javax.persistence.Entity;
import javax.persistence.Id;

@Entity
public class Employee // Entity class
{
    @Id
    private int eid;
    private String ename;
    private double salary;
    private String designation;

    public int getEid() {
        return eid;
    }
    public void setEid(int eid) {
        this.eid = eid;
    }
    public String getEname() {
        return ename;
    }
    public void setEname(String ename) {
        this.ename = ename;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
    public String getDesignation() {
        return designation;
    }
    public void setDesignation(String designation) {
        this.designation = designation;
    }

}
```

- **Test class for inserting an object into database.**

```java
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.EntityTransaction;
import javax.persistence.Persistence;

public class InsertEmployee {

    public static void main(String[] args) {
        // create EntityManagerFactory
        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        // create EntityManager
        EntityManager manager = factory.createEntityManager();
        // create EntityTransaction
        EntityTransaction transaction = manager.getTransaction();

        // create Employee object
        Employee e = new Employee();
        // set values to employee object
        e.setEid(2);
        e.setEname("shivu");
        e.setSalary(420000);
        e.setDesignation("manager");

        transaction.begin();

        manager.persist(e);

        transaction.commit();

    }
}
```

- **Test class for searching an object from database based on primary key.**

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class SearchingEmployee {

    public static void main(String[] args) {

        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        EntityManager manager = factory.createEntityManager();

        // no modification in table , means no transaction happening.so no entity transaction

         Employee e = manager.find(Employee.class , 5);

         if(e !=  null)
         {
             System.out.println("Employee details are : ");
             System.out.println("EMP ID : "+e.getEid());
             System.out.println("EMP NAME : "+e.getEname());
             System.out.println("EMP DESIGNATION : "+e.getDesignation());
             System.out.println("EMP SALARY : "+e.getSalary());
         }
         else
         {
             System.out.println("ID NOT FOUND..");
         }

    }
}
```

- **Test class for removing/deleting an object from database.**

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.EntityTransaction;
import javax.persistence.Persistence;

public class DeletingEmployee {

    public static void main(String[] args) {

        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        EntityManager manager = factory.createEntityManager();
        EntityTransaction transaction = manager.getTransaction();

        Employee e = manager.find(Employee.class, 1);

        if(e != null)
        {
                transaction.begin();
                manager.remove(e);
                transaction.commit();
        }
        else
        {
            System.out.println("ID not found.");
        }
    }
}
```

- **Test class to update an object in database.**

```
public class UpdatingEmployeeObject {

    public static void main(String[] args) {

        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        EntityManager manager = factory.createEntityManager();
        EntityTransaction  transaction = manager.getTransaction();

        Employee emp = manager.find(Employee.class, 5);
        if(emp != null) {
            emp.setSalary(150000);
            emp.setDesignation("senior developer");

            transaction.begin();
            manager.merge(emp);
            transaction.commit();
        }
        else {
            System.out.println("employee id doesnt exist...");
        }
    }
}
```

- **Test class for select HQL query.**

```
public class DisplayAllEmployeeUsingHQL {

    public static void main(String[] args) {
        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        EntityManager manager = factory.createEntityManager();

        Query q = manager.createQuery("select e from Employee e");

        List<Employee> list = q.getResultList();

        if (list.isEmpty()) {
            System.out.println("Employee details not found..");
        }

        else {
            for (Employee emp : list) {
                System.out.println("EMPLOYEE ID : " + emp.getEmpid());
                System.out.println("NAME : " + emp.getName());
                System.out.println("DESIGNATION : " + emp.getDesignation());
                System.out.println("SALARY : " + emp.getSalary());
                System.out.println();
            }
        }
    }
}
```

- **Test class for select HQL query based on condition.**

```
public class DisplayBasedOnSalaryUsingHQL {

    public static void main(String[] args) {
        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        EntityManager manager = factory.createEntityManager();

        Query q = manager.createQuery("select e from Employee e where e.salary > 50000");

        List<Employee> list = q.getResultList();

        if(list.isEmpty()) {
            System.out.println("no employee details found");
        }
        else {
            for(Employee emp : list) {
                System.out.println("EMPLOYEE NAME : "+emp.getName()+"\nDESIGNATION : "+emp.getDesignation());
            }
        }
    }
}
```

- **Test class for delete HQL query.**

```
import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.EntityTransaction;
import javax.persistence.Persistence;
import javax.persistence.Query;

public class DeleteUisngHql {

    public static void main(String[] args) {

        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        EntityManager manager = factory.createEntityManager();
        EntityTransaction transaction = manager.getTransaction();

        // to delete objects based on ename attribute
        Query q = manager.createQuery("delete from Employee e where e.ename ='mala' ");
        transaction.begin();
        q.executeUpdate();
        transaction.commit();
    }
}
```

- **Test class for update HQL query.**

```
public class UpdateEmployeeByNameUsingHQL {

    public static void main(String[] args) {

        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        EntityManager manager = factory.createEntityManager();
        EntityTransaction transaction = manager.getTransaction();

        Query q = manager.createQuery("update Employee e set e.designation='manager' , e.salary=90000 where e.name='mala' ");

        transaction.begin();
        int rows = q.executeUpdate();
        transaction.commit();

        if(rows == 0) {
            System.out.println("Invalid employee name. Updation failed.");
        }
        else {
            System.out.println("Employee details updated...");
        }
    }
}
```

• Test class for update HQL query by taking input from user.

```
public class DynamicallyUpdateEmployeeUsingHQL {

    public static void main(String[] args) {

        EntityManagerFactory factory = Persistence.createEntityManagerFactory("dev");
        EntityManager manager = factory.createEntityManager();
        EntityTransaction transaction = manager.getTransaction();

        // create dynamic HQL query with place holder and position
        Query q = manager.createQuery("update Employee e set e.salary=?1 where e.designation=?2 ");

        // take input from user
        Scanner sc = new Scanner(System.in);
        System.out.println("enter then designation whose salary you want to update : ");
        String designation = sc.next();
        System.out.println("enter the new salary : ");
        double salary = sc.nextDouble();

        // set user input in position of place holders
        q.setParameter(1, salary);
        q.setParameter(2, designation);

        // execute the query
        transaction.begin();
        int rows = q.executeUpdate();
        transaction.commit();

        if (rows == 0) {
            System.out.println("Invalid designation.Updation failed...");
        } else {
            System.out.println("salary updated successfully....");
        }
    }
}
```